

SKILL_WEEK-04

(SESSION NUMBER - 7)

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

```
UW PICO 5.09
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent Process PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());
        printf("Child is performing its task...\n");

        sleep(3);

        printf("Child task completed.\n");

        exit(5);
    }
    else
    {
        printf("Parent created Child PID: %d\n", pid);
        printf("Parent is waiting for the child using waitpid()...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child terminated normally.\n");
            printf("Child Exit Status: %d\n", WEXITSTATUS(status));
        }
        else
        {
            printf("Child terminated abnormally.\n");
        }

        printf("Parent process resumes execution.\n");
    }

    return 0;
}
```

Fig 1: C program demonstrating process synchronization and child monitoring using waitpid()

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill7_task1.c -o bin/skill7_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill7_task1
Parent Process PID: 66689
Parent created Child PID: 66690
Parent is waiting for the child using waitpid()...
Child Process PID: 66690
Child is performing its task...
Child task completed.
Child terminated normally.
Child Exit Status: 5
Parent process resumes execution.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Execution output showing parent-child synchronization and child exit status using waitpid()

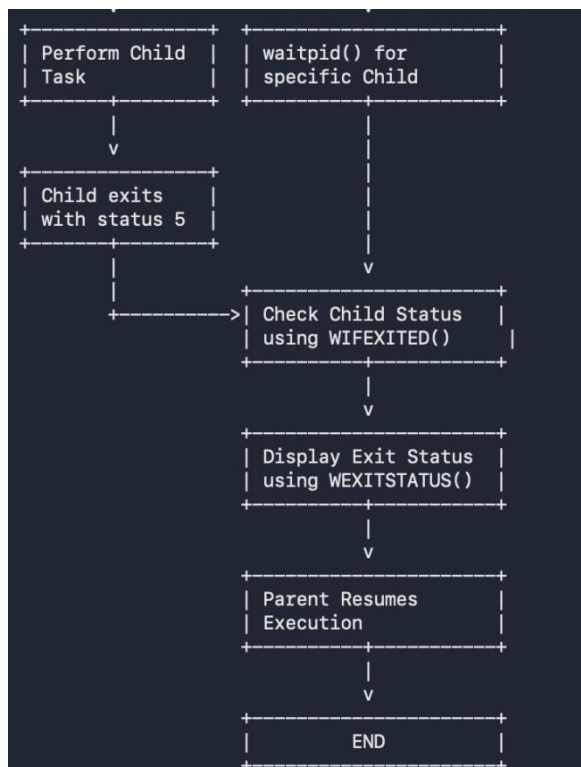
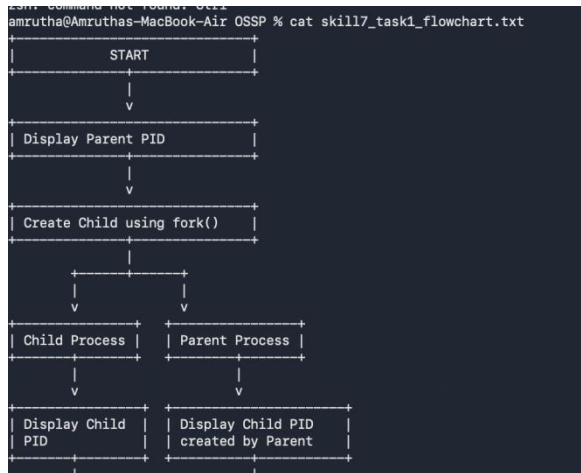


Fig 3: Flowchart for process synchronization and child monitoring using waitpid()

Result:

The child process was created successfully using fork() and monitored using waitpid(). The parent process waited for the specific child to complete, retrieved its exit status, and resumed execution after successful synchronization.

Task 2: Retrieve the PATH variable, parse search directories, locate executables, verify permissions, handle missing commands, and test command resolution.

```
UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_PATH 1024

int main()
{
    char command[100];
    char *path_env;
    char path_copy[MAX_PATH];
    char *directory;
    char full_path[MAX_PATH];

    path_env = getenv("PATH");

    if (path_env == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH Variable:\n%s\n", path_env);

    printf("Enter command to search: ");
    scanf("%99s", command);

    strncpy(path_copy, path_env, sizeof(path_copy) - 1);
    path_copy[sizeof(path_copy) - 1] = '\0';

    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        snprintf(full_path, sizeof(full_path),
                 "%s/%s", directory, command);

        printf("Checking: %s\n", full_path);

        if (access(full_path, X_OK) == 0)
        {
            printf("\nCommand found!\n");
            printf("Executable Path: %s\n", full_path);
            return 0;
        }

        directory = strtok(NULL, ":");
    }

    printf("\nCommand '%s' not found in PATH.\n", command);

    return 0;
}
```

Fig 4: C program for retrieving the PATH variable and resolving executable commands

Test Case - 1:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill7_task2.c -o bin/skill7_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill7_task2
PATH Variable:
/opt/homebrew/opt/openssl@1.1/bin:/opt/homebrew/opt/postgresql@17/bin:/opt/homebrew/bin:/opt/homebrew/sbin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/pkg/env/global/bin:/opt/homebrew/bin
Enter command to search: ls
Checking: /opt/homebrew/opt/openssl@1.1/bin/ls
Checking: /opt/homebrew/opt/postgresql@17/bin/ls
Checking: /opt/homebrew/bin/ls
Checking: /opt/homebrew/sbin/ls
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/ls
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/ls
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/ls
Checking: /usr/local/bin/ls
Checking: /System/Cryptexes/App/usr/bin/ls
Checking: /usr/bin/ls
Checking: /bin/ls
Command found!
Executable Path: /bin/ls
```

Fig 5: Successful resolution of the ls command using the PATH environment variable

Test case - 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill7_task2
PATH Variable:
/opt/homebrew/opt/openssl@1.1/bin:/opt/homebrew/opt/postgresql@17/bin:/opt/homebrew/bin:/opt/homebrew/sbin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/pkg/env/global/bin:/opt/homebrew/bin
Enter command to search: gcc
Checking: /opt/homebrew/opt/openssl@1.1/bin/gcc
Checking: /opt/homebrew/opt/postgresql@17/bin/gcc
Checking: /opt/homebrew/bin/gcc
Checking: /opt/homebrew/sbin/gcc
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/gcc
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/gcc
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/gcc
Checking: /usr/local/bin/gcc
Checking: /System/Cryptexes/App/usr/bin/gcc
Checking: /usr/bin/gcc
Command found!
Executable Path: /usr/bin/gcc
```

Fig 6: Resolving the compiler executable using PATH directory search

Test case - 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill7_task2
PATH Variable:
/opt/homebrew/opt/openssl@1.1/bin:/opt/homebrew/opt/postgresql@17/bin:/opt/homebrew/bin:/opt/homebrew/sbin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/Library/Frameworks/Python.framework/Versions/3.14/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/pkg/env/global/bin:/opt/homebrew/bin
Enter command to search: abcxyzcommand
Checking: /opt/homebrew/opt/openssl@1.1/bin/abcxyzcommand
Checking: /opt/homebrew/opt/postgresql@17/bin/abcxyzcommand
Checking: /opt/homebrew/bin/abcxyzcommand
Checking: /opt/homebrew/sbin/abcxyzcommand
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/abcxyzcommand
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/abcxyzcommand
Checking: /Library/Frameworks/Python.framework/Versions/3.14/bin/abcxyzcommand
Checking: /usr/local/bin/abcxyzcommand
Checking: /System/Cryptexes/App/usr/bin/abcxyzcommand
Checking: /usr/bin/abcxyzcommand
Checking: /bin/abcxyzcommand
Checking: /usr/sbin/abcxyzcommand
Checking: /sbin/abcxyzcommand
Checking: /var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin/abcxyzcommand
Checking: /var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin/abcxyzcommand
Checking: /var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin/abcxyzcommand
Checking: /pkg/env/global/bin/abcxyzcommand
Checking: /opt/homebrew/bin/abcxyzcommand
Command 'abcxyzcommand' not found in PATH.
```

Fig 7: Handling a command that is not present in the PATH directories

```

amrutha@Amruthas-MacBook-Air OSSP % nano skill7_task2_flowchart.txt
amrutha@Amruthas-MacBook-Air OSSP % cat skill7_task2_flowchart.txt
+-----+
| START |
+-----+
|       |
| v     |
+-----+
| Retrieve PATH using getenv() |
+-----+
|       |
| v     |
+-----+
| Read Command Name |
+-----+
|       |
| v     |
+-----+
| Split PATH using ':' |
+-----+
|       |
| v     |
+-----+
| Build Full Path |
| directory/command |
+-----+
|       |
| v     |
+-----+
| Check Execute Permission |
| using access(..., X_OK) |
+-----+

```

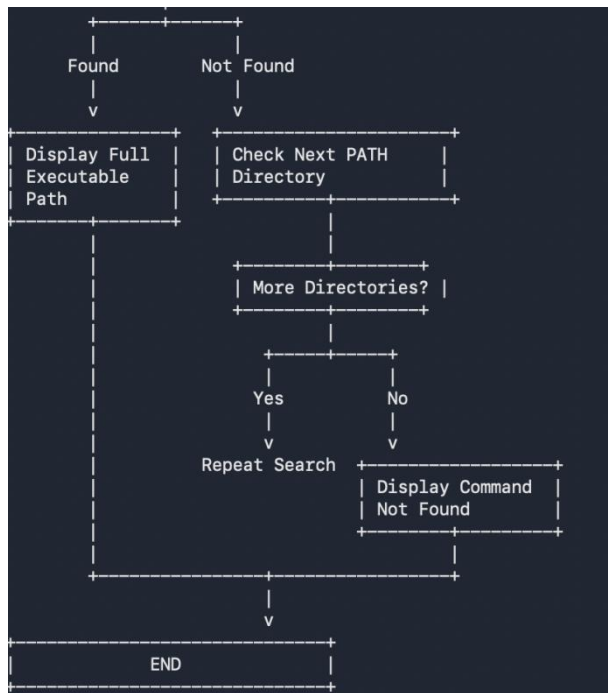


Fig 8: Flowchart for PATH variable parsing and executable command resolution

Result:

The PATH environment variable was retrieved and parsed successfully. Executable commands were located by searching PATH directories and checking execution permission using `access()`. Missing commands were also handled correctly.

(SESSION NUMBER - 8)

Task 1: To Detect variable references, expand values, handle undefined variables, update tokens, support nested variables, and test expansion logic.

```
UW PICO 5.09

    if (k == 0)
    {
        output[j++] = '$';
        continue;
    }

    char *value = getenv(var_name);

    if (value != NULL)
    {
        for (int m = 0; value[m] != '\0'; m++)
        {
            output[j++] = value[m];
        }
    }
    else
    {
        printf("Warning: Undefined variable %s\n", var_name);
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char expanded[MAX_OUTPUT];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, expanded);

    printf("\nOriginal Command : %s\n", input);
    printf("Expanded Command : %s\n", expanded);

    return 0;
}
```

Fig 1: C program for detecting and expanding environment variable references

Test Case - 1:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill8_task1.c -o bin/skill8_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill8_task1
Enter command: echo $USER

Original Command : echo $USER
Expanded Command : echo amrutha
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Expansion of the USER environment variable in the command

Test Case - 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill8_task1
Enter command: echo $HOME/Documents

Original Command : echo $HOME/Documents
Expanded Command : echo /Users/amrutha/Documents
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 3: Expansion of the HOME environment variable while preserving additional path text

Test Case - 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill8_task1
Enter command: echo $USER $HOME

Original Command : echo $USER $HOME
Expanded Command : echo amrutha /Users/amrutha
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 4: Expansion of multiple environment variables in a single command

Test Case - 4:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill8_task1
Enter command: echo $MYUNKNOWNVAR
Warning: Undefined variable $MYUNKNOWNVAR

Original Command : echo $MYUNKNOWNVAR
Expanded Command : echo
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 5: Handling an undefined environment variable during expansion

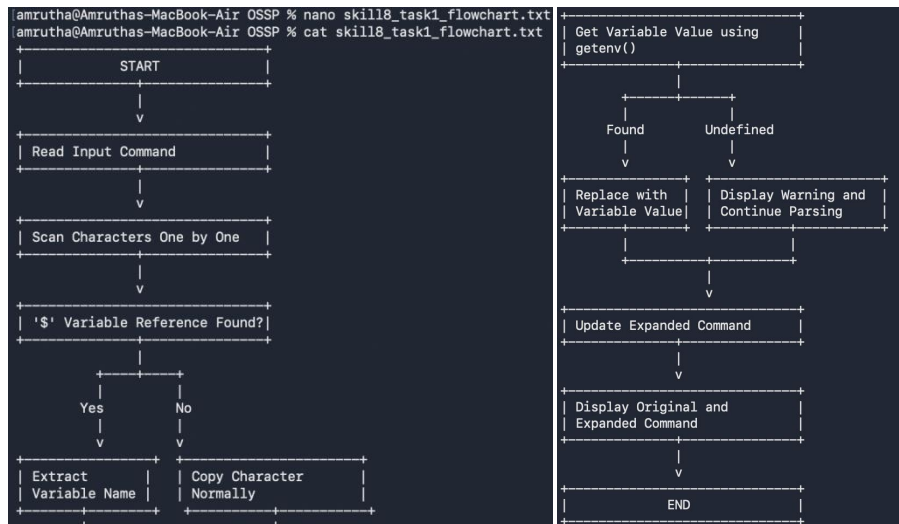


Fig 6: Flowchart for detecting and expanding environment variable references

Result:

Environment variable references were detected and expanded successfully. Variables such as \$USER and \$HOME were replaced with their corresponding values, multiple variables were handled correctly, and undefined variables were identified safely.

Task 2: Identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain state, and test dispatch logic.

```

UW PICO 5.09
int main()
{
    char command[100];
    char argument[200];

    while (1)
    {
        printf("\nmini-shell> ");
        scanf("%99s", command);

        if (strcmp(command, "pwd") == 0)
        {
            builtin_pwd();
        }
        else if (strcmp(command, "cd") == 0)
        {
            scanf("%199s", argument);
            if (chdir(argument) == 0)
            {
                printf("Directory changed successfully.\n");
            }
            else
            {
                perror("cd failed");
            }
        }
        else if (strcmp(command, "help") == 0)
        {
            builtin_help();
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exiting mini shell...\n");
            break;
        }
        else
        {
            printf("Invalid built-in command: %s\n", command);
        }
    }

    return 0;
}

```

Fig 7: C program implementing built-in commands and command dispatch logic

Test Case - 1:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill8_task2.c -o bin/skill8_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill8_task2

mini-shell> pwd
Current Directory: /Users/amrutha/OSSP
```

Fig 8: Execution of the built-in pwd command to display the current working directory

Test Case - 2:

```
mini-shell> help
Available Built-in Commands:
pwd - Display current directory
cd - Change current directory
help - Display help information
exit - Exit the program
```

Fig 9: Display of supported built-in commands using the help command

Test Case - 3:

```
mini-shell> cd ..
Directory changed successfully.

mini-shell> pwd
Current Directory: /Users/amrutha
```

Fig 10: Changing and maintaining the current working directory using the built-in cd command

Test Case - 4:

```
mini-shell> hello
Invalid built-in command: hello
```

Fig 11: Handling an invalid built-in command in the mini shell

Test Case - 5:

```
mini-shell> exit
Exiting mini shell...
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 12: Termination of the mini shell using the built-in exit command

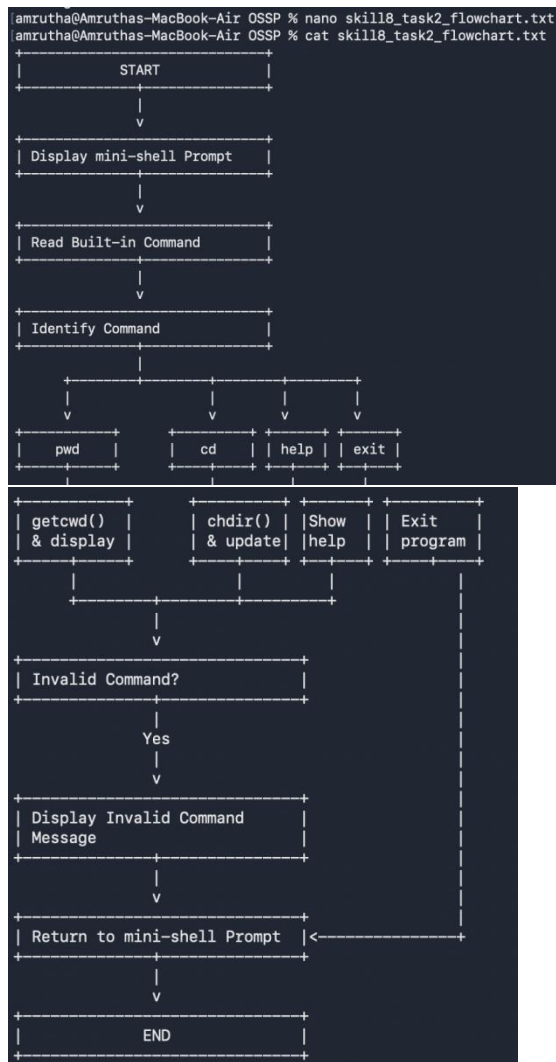


Fig 13: Flowchart for identifying and executing built-in commands in the mini shell

Result:

Built-in commands such as pwd, cd, help, and exit were implemented successfully. Commands were identified and executed within the same process, directory state was maintained using cd, and invalid commands were handled correctly.